

Model for Math Problems Difficulty Prediction

Machine Learning Project Report

Matus Bucher
bucher5@uniba.sk

Comenius University in Bratislava,
Faculty of Mathematics, Physics and Informatics

GitHub repository: github.com/matusbucher/ml-project

1 Introduction

In this project, we implement and evaluate a model for predicting the difficulty of mathematical problems using a range of preprocessing techniques and machine learning methods. Then we compare the performance of these modeling approaches. Everything was implemented in Python using libraries Numpy [4], Scikit-learn [7], Textstat [1], PyTorch [6] and Transformers [9].

We used publicly available datasets of mathematical problems (spanning areas such as algebra, calculus, geometry, combinatorics, etc.), which are usually meant for reasoning language models. These datasets often include estimated difficulty annotations, which we use as supervision for model training.

Accurate difficulty prediction (not only for mathematical problems, but generally for any task) has several practical applications, including exam difficulty assessment, analytics for instructors and curriculum designers, adaptive learning platforms, student study planning through time-on-task estimation, and benchmarking models that solve mathematical problems.

2 Datasets

We selected two datasets that include explicit difficulty annotations, but in different formats.

- **MATH-lighteval** [5] ([link](#))
 - **rows:** 12,500
 - **difficulty label:** *level* – integer number from 1 to 5 (level 1 means the lowest difficulty)
 - **other features:** solution description; problem type (algebra, counting & probability, geometry, intermediate algebra, number Theory, prealgebra, precalculus)

- **Easy2Hard-Bench** (E2H-AMC subset) [3] ([link](#))
 - **rows:** 3,980
 - **difficulty label:** *rating* – float number in the range $(0, 1)$ (the smaller number the lower difficulty)
 - **other features:** numerical answer; solution description; contest metadata (name, year, contest type); auxiliary difficulty estimates (e.g., standard deviation, unnormalized rating, lower/upper bounds suggested by AoPS)

In both datasets, problem statements are provided as text in $\text{\TeX}$ format, describing the problem in natural language (English) and mathematical formulas.

Each dataset also includes solution descriptions (also in text representation), which can optionally be used as model inputs. However, we did not experiment with this option in our project. Even in practical scenario, difficulty estimation should ideally rely solely on the problem statement, without requiring access to a solution. Still, we used it for one of the baseline models (see section 4).

For all model training, we used an 80 : 20 train-test split. Together with filtering out invalid samples (e.g., missing difficulty labels), the resulting dataset sizes were as follows:

- **MATH-lighteval**
 - train: 9,998
 - test: 2,500
- **Easy2Hard-Bench**
 - train: 3,180
 - test: 795

3 Model Description

Based on the selected datasets, the primary input to our model is the problem statement provided as raw text, including $\text{\TeX}$ notation.

The model outputs a single continuous value in the range $[0, 1]$, representing the estimated difficulty of a problem, where 0 corresponds to the lowest difficulty and 1 to the highest difficulty. This unified representation enables combining the two datasets (see section 9). It also allows for easier evaluation of model performance (e.g., using RMSE metric).

Accordingly, the discrete *level* label in the MATH-lighteval dataset needs to be mapped to $[0, 1]$. In order to avoid having large number of marginal labels in training data, we use linear normalization from $[1, 5]$ to $[0.1, 0.9]$ (level 1 maps to 0.1, level 5 maps to 0.9) as follows:

$$\text{normalize}(l) = 0.1 + 0.2(l - 1)$$

The *rating* label in the Easy2Hard-Bench dataset is already in normalized format and does not require additional transformation.

The continuous output implies a regression task. Since we defined the output range to be $[0, 1]$, we need to ensure that the model predictions also lie within this interval. Therefore, we apply a bounding function to the model outputs – but only at evaluation time, not during training. We used a simple clipping:

$$\text{clip}(x) = \min(\max(x, 0), 1)$$

Suitable alternative would be a sigmoid function, but we found clipping to be sufficient for our purposes.

4 Baseline Models

We considered four simple baseline models to establish reference performance levels:

- **Random Model** – Predicts a difficulty value sampled uniformly at random from the interval $(0, 1)$.
- **Average Model** – Predicts the mean difficulty value computed from the training data.
- **Description Length Model** – A linear regression model using the length of the problem description as the sole feature, with predictions bounded to $[0, 1]$ ¹.
- **Solution Length Model** – A linear regression model using the length of the solution description as the sole feature, with predictions bounded to $[0, 1]$.

In the table 1 and the table 2, we can see measured scores of each model on the two datasets. We chose two metrics – R-squared (R^2) and root mean squared error (RMSE).

Table 1: Scores for baseline models on MATH-lighteval

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Random	-0.955	0.452	-0.981	0.454
Average	0.000	0.323	0.000	0.323
Description Length	0.059	0.313	0.061	0.313
Solution Length	0.210	0.287	0.210	0.287

¹ Bounding is applied only at evaluation time, not during training.

Table 2: Scores for baseline models on Easy2Hard-Bench

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Random	-1.485	0.385	-1.430	0.382
Average	0.000	0.244	-0.002	0.245
Description Length	0.001	0.244	-0.005	0.246
Solution Length	0.161	0.224	0.124	0.229

Among the baseline models, the solution length model achieves the highest accuracy. However, it relies on an optional feature; therefore, the description length model is still relevant for later comparisons, as it uses the same input feature as our later models.

5 Preprocessing Approaches

The primary input to our model is a natural language text, but it also contains mathematical notation in $\text{\TeX}$ format, which might or might not carry useful information for difficulty prediction. Therefore, when implementing our models, we experimented with different preprocessing strategies:

- (a) **keep everything** – The raw text is used as-is, including all $\text{\TeX}$ commands and mathematical symbols.
- (b) **remove $\text{\TeX}$** – All $\text{\TeX}$ commands and mathematical symbols are stripped from the text, including whole equations and environments. This leaves only the natural language parts of the problem statement.
- (c) **remove $\text{\TeX}$ symbols** – Only some $\text{\TeX}$ symbols are removed (e.g., $\backslash$, $\$$, $\{ \}$, $\dots$), while keeping the rest of the text intact. This keeps numbers and expressions, but doesn't remove $\text{\TeX}$ commands (this might be desired).

In the following sections, we will refer to these approaches as (a), (b), and (c).

Apart from these three approaches, we applied standard preprocessing steps, including lowercasing, inserting spaces (between mathematical operators) and normalizing whitespace (collapsing multiple whitespaces into one space).

6 TF-IDF with Linear Regression

For a starting point, we chose a simple approach combining *TF-IDF* text vectorization with a linear regression model. The reason behind choosing a linear model is that TF-IDF produces high-dimensional sparse vectors, which linear models can handle well.

We tried two different linear models – *Ridge regression* and *SVR* (Support Vector Regressor) *with linear kernel*. We also experimented with hyperparameters of both the vectorizer and the regression models using grid search with 5-fold cross-validation and negative mean squared error as the estimator. We utilized Scikit-learn’s [7] `TfidfVectorizer`, `Ridge`, and `SVR` implementations, as well as `GridSearchCV` for hyperparameter tuning.

6.1 Hyperparameter Optimization

For `Ridge`, we searched over `alpha` – regularization strength. For `SVR`, we searched over `C` (regularization parameter) and `epsilon` (specifies the epsilon-tube). For `TfidfVectorizer`, we searched over the following hyperparameters:

- `min_df` – Minimal document occurrence for a token to be kept.
- `max_df` – Maximal document frequency (percentage) for a token to be kept.

Aside from those, we fixed the following settings for `TfidfVectorizer`:

- `ngram_range=(1,2)` – Using unigrams and bigrams. Captures fixed phrases such as “partial derivative” or “solve for”.
- `sublinear_tf=True` – Apply sublinear term frequency scaling, which improves robustness for long statements.
- `norm="l2"` – Apply L2 normalization to the resulting vectors.
- `stop_words="english"` – Remove English stop words.

We evaluated the models and preprocessing approaches (a), (b), and (c). TF-IDF works straight on raw text, therefore we don’t need any further text processing. Since there are huge number of combinations “settings”, we searched for the best hyperparameters separately for vectorizer and regressor by fixing one and searching for the other (we have done multiple runs with different “starting point”). Therefore, the found best hyperparameters may not be globally optimal, but they are close to optimal in terms of performance.

In results from the grid search, the optimal values for hyperparameters were always or almost always same (there weren’t various combinations of vectorizer and regressor hyperparameter values). In the following, we list values we searched over together with the best found:

- `TfidfVectorizer.min_df`: { 1, 3, 5, 10, 15 } – best: 3
- `TfidfVectorizer.max_df`: { 0.1, 0.3, 0.5, 0.7, 0.9 } – best: 0.3
- `Ridge.alpha`: { 0.1, 1.0, 10.0 } – best: 1.0
- `SVR.C`: { 0.01, 0.1, 1.0, 10.0 } – best: 0.1
- `SVR.epsilon`: { 0.01, 0.1, 1.0 } – best: 0.1

The value of `min_df` indicates that removing very rare tokens improves model performance, while the value of `max_df` suggests that removing common tokens (appearing in more than 30% of documents) is also beneficial. This is quite expected result, although the `max_df` value is a bit lower than usual (the reason is probably common mathematical language).

6.2 Results

For final evaluation of models, we fixed hyperparameters to the found best values. In the table 3 and the table 4, we can see the train and test scores with all three preprocessing approaches.

Table 3: Scores for TF-IDF + Ridge and SVR on MATH-lighteval

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Ridge (a)	0.627	0.158	0.328	0.212
Ridge (b)	0.552	0.173	0.298	0.216
Ridge (c)	0.625	0.158	0.328	0.212
SVR (a)	0.512	0.181	0.326	0.212
SVR (b)	0.451	0.192	0.294	0.217
SVR (c)	0.511	0.181	0.325	0.212

Table 4: Scores for TF-IDF + Ridge and SVR on Easy2Hard-Bench

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Ridge (a)	0.815	0.105	0.609	0.153
Ridge (b)	0.775	0.116	0.545	0.165
Ridge (c)	0.812	0.106	0.603	0.154
SVR (a)	0.729	0.127	0.595	0.156
SVR (b)	0.688	0.136	0.526	0.169
SVR (c)	0.725	0.128	0.592	0.156

From the results, we can see that both Ridge and SVR perform best with preprocessing approaches (a) and (c), while approach (b) yields slightly worse results. This suggests that retaining mathematical symbols in the text provides useful information for difficulty prediction. The difference between (a) and (c) is negligible, indicating that removing certain $\text{\TeX}$ symbols does not significantly impact performance.

Overall, the best performing model on both datasets is TF-IDF combined with Ridge regression using preprocessing approach (a). Compared against the baseline models, especially the description length and solution length models, the TF-IDF + Ridge model has the following error reductions on the test sets (we are considering only RMSE scores here):

- **Description Length Baseline vs. TF-IDF + Ridge**
 - MATH-lighteval: $0.313 \rightarrow 0.212 - \approx 32.27\%$ error reduction
 - Easy2Hard-Bench: $0.246 \rightarrow 0.153 - \approx 37.8\%$ error reduction
- **Solution Length Baseline vs. TF-IDF + Ridge**
 - MATH-lighteval: $0.287 \rightarrow 0.212 - \approx 26.13\%$ error reduction
 - Easy2Hard-Bench: $0.229 \rightarrow 0.153 - \approx 33.19\%$ error reduction

7 Linguistic Features with Tree-Based Model

Another approach we experimented with is using linguistic features extracted from the solution description, combined with a tree-based regression model. This choice comes from the fact that linguistic feature vector is low-dimensional and dense, making it suitable for such models.

7.1 Linguistic Features Set

We used Textstat library [1] for linguistic features extraction together with some own implemented features. The final feature set includes:

- **sentence count**
- **lexicon (words) count**
- **syllable count**
- **Flesch reading ease**
- **Flesch-Kincaid grade level**
- **Dale-Chall readability score**
- **difficult words count** – Textstat’s `difficult_words` function, estimates the number of challenging or complex words
- **math expression count** – detected via $\text{\TeX}$ math environments
- **numbers count** – number of “numerical words” in the text
- **operators count** – number of mathematical operators (e.g., $+$, $-$, $/$, ...)
- **variables count** – number of isolated letters inside math expressions

Since feature extraction makes up the whole preprocessing step, we don’t need to consider approaches from the section 5 here. However, we used description without $\text{\TeX}$ (approach (b)) for almost all feature extraction functions, except for math expression count, numbers count, operators count, and variables count, which require $\text{\TeX}$ math environments to be present.

7.2 Regression Models and Hyperparameter Optimization

As with TF-IDF model, we compare two different regression models – *Random Forest Regressor* and *Gradient Boosting Regressor*. Since these models have several hyperparameters, we performed corss-validated grid search (5-fold, negative mean squared error). We use `RandomForestRegressor`, `GradientBoostingRegressor`, and once again `GridSearchCV` from Scikit-learn library [7].

We list the hyperparameters values we searched over below, as well as ones we fixed:

- `RandomForestRegressor`
 - fixed: `bootstrap=True`
 - searched:
 - * `n_estimators`: { 300, 500, 700 }
 - * `max_depth`: { 5, 10, 20 }
 - * `min_samples_split`: { 10, 20, 50 }
 - * `min_samples_leaf`: { 3, 5, 10 }
 - * `max_features`: { 0.6, 0.8, 1.0 }
- `GradientBoostingRegressor`
 - fixed: `loss="squared_error"`, `max_features=None`
 - searched:
 - * `n_estimators`: { 300, 500, 700 }
 - * `learning_rate`: { 0.01, 0.1, 0.5 }
 - * `max_depth`: { 5, 10, 20 }
 - * `min_samples_split`: { 10, 20, 50 }
 - * `min_samples_leaf`: { 3, 5, 10 }
 - * `subsample`: { 0.6, 0.8, 1.0 }

Unlike with TF-IDF model, the found values varied between datasets (partially because of different datasets sizes). Their are shown in the table 5 and in the table 6.

Table 5: `RandomForestRegressor` hyperparameters for each dataset

Hyperparameter	MATH-lighteval	Easy2Hard-Bench
<code>n_estimators</code>	300	500
<code>max_depth</code>	20	10
<code>min_samples_split</code>	10	10
<code>min_samples_leaf</code>	3	3
<code>max_features</code>	0.6	0.8

Table 6: GradientBoostingRegressor hyperparameters for each dataset

Hyperparameter	MATH-lighteval	Easy2Hard-Bench
n_estimators	300	700
learning_rate	0.01	0.01
max_depth	10	5
min_samples_split	20	50
min_samples_leaf	3	10
subsample	0.6	0.8

7.3 Results

The evaluated scores of models with the found best hyperparameters are shown in the table 7 and in the table 8.

Table 7: Scores for Ling. Feature + Random Forest and Gradient Boosting on MATH-lighteval

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Random Forest	0.654	0.152	0.295	0.217
Gradient Boosting	0.547	0.174	0.297	0.217

Table 8: Scores for Ling. Feature + Random Forest and Gradient Boosting on Easy2Hard-Bench

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Random Forest	0.670	0.140	0.444	0.183
Gradient Boosting	0.606	0.153	0.457	0.180

Both models achieve very similar performance on both datasets. Maybe Random Forest is slightly overfitting, which we can see from the train vs. test scores. Therefore we take the Gradient Boosting model as the representative one. Compared against the baseline models (description length and solution length), the error reductions on the test sets are as follows:

- **Description Length Baseline vs. LingFeature + Gradient Boosting**
 - MATH-lighteval: $0.313 \rightarrow 0.217 - \approx 30.67\%$ error reduction
 - Easy2Hard-Bench: $0.246 \rightarrow 0.18 - \approx 26.83\%$ error reduction

– **Solution Length Baseline vs. LingFeature + Gradient Boosting**

- MATH-lighteval: $0.287 \rightarrow 0.217 - \approx 24.39\%$ error reduction
- Easy2Hard-Bench: $0.229 \rightarrow 0.18 - \approx 21.4\%$ error reduction

This model performs worse than the TF-IDF + Ridge model from the previous section. Of course, the choice of linguistic features is very important here – different or additional features could improve the performance.

7.4 Feature Importances

This approach still gives us some useful information, since the features are interpretable. We can retrieve feature importances from the trained models (Scikit-learn implementations provide this functionality out-of-the-box). In the table 9, we list feature importances for all four trained models and their average importance. In the table, we denote

- M_{RF} – Random Forest on MATH-lighteval
- M_{GB} – Gradient Boosting on MATH-lighteval
- E_{RF} – Random Forest on Easy2Hard-Bench
- E_{GB} – Gradient Boosting on Easy2Hard-Bench

Table 9: Feature importances for tree-based models M_{RF} , M_{GB} , E_{RF} , and E_{GB}

Feature	M_{RF}	M_{GB}	E_{RF}	E_{GB}	Average
sentence c.	0.0134	0.0117	0.0140	0.0137	0.0132
lexicon c.	0.0904	0.0674	0.0468	0.0396	0.0611
syllable c.	0.1494	0.1507	0.0591	0.0629	0.1055
FRE	0.0909	0.0949	0.0531	0.0451	0.0710
FKGL	0.0980	0.1016	0.0995	0.0983	0.0994
Dale-Chall	0.1248	0.1395	0.1227	0.1305	0.1294
difficult w. c.	0.1014	0.1009	0.0375	0.0289	0.0672
math expr. c.	0.0809	0.0826	0.0725	0.0768	0.0782
numbers c.	0.0640	0.0633	0.0609	0.0552	0.0608
operators c.	0.0947	0.0990	0.0751	0.0353	0.0760
variables c.	0.0922	0.0884	0.3587	0.4134	0.2382

From the table, we can see that the most important features (on average) are variables count, Dale-Chall readability score and syllable count (which is highly correlated with text length). On the other hand, sentence count are by far the least important feature. We can also observe differences between datasets – for example, variables count is much more important for Easy2Hard-Bench dataset than for MATH-lighteval.

8 Transformer Model

For our final approach, we implemented a transformer-based regression model using the HuggingFace Transformers library [9] together with PyTorch [6]. We used pre-trained language models and fine-tuned them for our regression task. We chose two different models:

- **BERT-base** [2] – General-purpose language model pre-trained on large number of English texts.
- **MathBERT** [8] – Domain-specific variant of BERT pre-trained on mathematical texts (which is more suitable for our task).

The architecture consists of a transformer encoder (either BERT-base or MathBERT) with a regression head. The implementation uses the following components:

1. **tokenizer** – Converts input text into token IDs using the pre-trained tokenizer corresponding to the selected model.
2. **transformer encoder** – BERT-base or MathBERT, both with a regression head (single output neuron).
3. **regression head** – The final layer outputs a real value, interpreted as the predicted difficulty.
4. **bounding function** – At evaluation, predictions are clipped to $[0, 1]$.

For the implementation, we used the API from the Transformers library [9], especially these classes:

- **AutoTokenizer** – tokenization (parameters described later)
- **AutoModelForSequenceClassification** – loading pre-trained models with regression head (`num_labels=1`, `problem_type="regression"`)
- **TrainingArguments** – specifying training hyperparameters (parameters described later)
- **Trainer** – training and evaluation (takes model, training arguments, datasets, and tokenizer as input)

Because of long training times, we didn't perform grid search with cross-validation here. Instead, we split the training data further into training and validation sets and monitored validation loss during training to prevent overfitting – we validated after end of each epoch, then loaded the model with the best validation loss. We also did two whole training iterations in order to set hyperparameter values at least somewhat optimally.

8.1 First Attempt

In the first attempt, we chose a 90 : 10 train-validation split. The dataset sizes for this attempt were:

- **MATH-lighteval**
 - training: 8,998
 - validation: 1,000
- **Easy2Hard-Bench**
 - training: 2,862
 - validation: 318

Then we chose hyperparameter values based on common recommendations for fine-tuning BERT models. We list hyperparameter values, which both models shared during training.

- **AutoTokenizer**
 - `max_length=256` – maximum number of tokens per input sequence
 - `padding=True` – pads short sequences
 - `truncation=True` – truncates long sequences
 - `return_tensors="pt"` – returns PyTorch tensors
- **TrainingArguments**
 - `num_train_epochs=4`
 - `per_device_train_batch_size=16`
 - `learning_rate=2e-5` – recommended for fine-tuning BERT
 - `weight_decay=0.01` – regularization (prevents overfitting)

The results of the first iteration are shown in the table 10 and in the table 11.

Table 10: Scores for Transformer models (BERT-base and MathBERT) on MATH-lighteval, first attempt

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
BERT-base	0.670	0.148	0.404	0.199
MathBERT	0.711	0.139	0.383	0.203

Table 11: Scores for Transformer models (BERT-base and MathBERT) on Easy2Hard-Bench, first attempt

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
BERT-base	0.877	0.086	0.738	0.125
MathBERT	0.884	0.083	0.701	0.134

As we can see, both models show a classic sign of overfitting – the training scores are significantly better than the testing scores – especially for the MathBERT model. It’s also interesting to note that the MathBERT model performed worse than the BERT-base model on the test sets (which was unexpected). This could be due to smaller dataset sizes as well as suboptimal hyperparameter choices. We tried to mitigate overfitting in the second attempt.

8.2 Second Attempt

First, we increased the size of the validation set to 20% of the original training data. The new dataset sizes were:

- **MATH-lighteval**
 - training: 7,998
 - validation: 2,000
- **Easy2Hard-Bench**
 - training: 2,544
 - validation: 636

Then we adjusted these parameters of `TrainingArguments`:

- `learning_rate=1e-5` – lowered learning rate for more stable training
- `weight_decay=0.1` – increased regularization (forcing model to generalize better)

We also increased dropout rate in hidden and attention layers from default 0.1 to 0.2, which can also help reduce overfitting. The results of the second iteration are shown in the table 12 and in the table 13.

Table 12: Scores for Transformer models (BERT-base and MathBERT) on MATH-lighteval, second attempt

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
BERT-base	0.416	0.198	0.352	0.208
MathBERT	0.44	0.193	0.351	0.208

Table 13: Scores for Transformer models (BERT-base and MathBERT) on Easy2Hard-Bench, second attempt

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
BERT-base	0.735	0.126	0.698	0.135
MathBERT	0.672	0.14	0.613	0.152

We can see that the overfitting issue has been mitigated – the difference between training and testing scores is much smaller now. However, the overall performance has decreased compared to the first attempt. This is likely due to reduced training data size and lower learning rate with not enough epochs. We could fix this in another attempt, but we left it as is due to time constraints.

While on MATH-lighteval dataset both models perform similarly, on Easy2Hard-Bench dataset the BERT-base model still outperforms the MathBERT model. This could be due to the fact that MathBERT was pre-trained on different types of mathematical texts than those present in our datasets, leading to suboptimal performance.

8.3 Results

Compared to previous models, both transformer models performed better, although not by a large margin. In the following, we report the error reduction on the test set against baseline models and the best TF-IDF + Ridge model from section 6. We will consider BERT-base from the first attempt as the representative one even though it might be overfitted, since it has the best test scores.

- **Description Length Baseline vs. BERT-base (1st attempt)**
 - MATH-lighteval: $0.313 \rightarrow 0.199 - \approx 36.42\%$ error reduction
 - Easy2Hard-Bench: $0.246 \rightarrow 0.125 - \approx 49.19\%$ error reduction
- **Solution Length Baseline vs. BERT-base (1st attempt)**
 - MATH-lighteval: $0.287 \rightarrow 0.199 - \approx 30.66\%$ error reduction
 - Easy2Hard-Bench: $0.229 \rightarrow 0.125 - \approx 45.41\%$ error reduction
- **TF-IDF + Ridge vs. BERT-base (1st attempt)**
 - MATH-lighteval: $0.212 \rightarrow 0.199 - \approx 6.13\%$ error reduction
 - Easy2Hard-Bench: $0.153 \rightarrow 0.125 - \approx 18.3\%$ error reduction

9 Combining Datasets

While datasets MATH-lighteval and Easy2Hard-Bench are similar in nature, they differ in difficulty distribution, problem descriptions, and other characteristics. However, the normalized difficulty representation allows us to combine datasets into a larger training set. When split with the same 80 : 20 train-test ratio, the combined dataset has the following sizes:

- **Combined (MATH-lighteval + Easy2Hard-Bench)**
 - training: 13,178
 - testing: 3,295

In this section, we present results of training models from previous sections on the combined dataset. We don't repeat the Transformer models here, since their training times are too long and we didn't really optimize their hyperparameters.

9.1 Baseline Models

The baseline models were re-evaluated on the combined dataset. The results are shown in the table 14.

Table 14: Baseline model performance on Combined Dataset

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Random	-1.286	0.398	-1.348	0.406
Average	0.000	0.264	0.000	0.265
Description Length	0.011	0.262	0.018	0.263
Solution Length	0.109	0.249	0.114	0.250

9.2 TF-IDF Models

For TF-IDF models, we used the same hyperparameter values as found in the section 6, since they were same for both datasets. We evaluated both Ridge and SVR models with all three preprocessing approaches. The results are shown in the table 15.

Table 15: Scores for TF-IDF + Ridge and SVR on combined dataset

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Ridge (a)	0.594	0.168	0.223	0.234
Ridge (b)	0.521	0.182	0.187	0.239
Ridge (c)	0.592	0.168	0.228	0.233
SVR (a)	0.470	0.192	0.229	0.233
SVR (b)	0.413	0.202	0.192	0.238
SVR (c)	0.469	0.192	0.234	0.232

We can see that the performance of TF-IDF models on the combined dataset is worse than on individual datasets (see table 3 and table 4). This is probably caused by increased variance in the combined data as well as different difficulty assessments. The best performing model is again TF-IDF + Ridge, but with preprocessing approach (c) (removing only $\text{\TeX}$ symbols), although the difference is tiny. For direct comparison, the error reductions against baseline models are as follows:

– **Description Length Baseline vs. TF-IDF + Ridge**

- Combined: $0.263 \rightarrow 0.233 - \approx 11.41\%$ error reduction

- (MATH-lighteval: $\approx 32.27\%$; Easy2Hard-Bench: $\approx 37.8\%$)
- **Solution Length Baseline vs. TF-IDF + Ridge**
- Combined: $0.250 \rightarrow 0.233 - \approx 6.8\%$ error reduction
 - (MATH-lighteval: $\approx 26.13\%$; Easy2Hard-Bench: $\approx 33.19\%$)

9.3 Linguistic Features Models

For linguistic features models, we had different hyperparameter values for individual datasets (see table 5 and table 6). Without running another grid search, we chose the values from MATH-lighteval because of its larger size (makes up $\approx 75\%$ of the combined dataset). The results are shown in the table 16.

Table 16: Scores for Ling. Feature + Random Forest and Gradient Boosting on combined dataset

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Random Forest	0.604	0.166	0.177	0.241
Gradient Boosting	0.446	0.196	0.183	0.240

Similarly to TF-IDF models, linguistic features models also perform worse on the combined dataset (for the same reasons). The Gradient Boosting model is slightly better on the test set, so we take it as the representative one. The error reductions against baseline models are as follows:

- **Description Length Baseline vs. LingFeature + Gradient Boosting**
- Combined: $0.263 \rightarrow 0.240 - \approx 8.74\%$ error reduction
 - (MATH-lighteval: $\approx 30.67\%$; Easy2Hard-Bench: $\approx 26.83\%$)
- **Solution Length Baseline vs. LingFeature + Gradient Boosting**
- Combined: $0.250 \rightarrow 0.240 - \approx 4\%$ error reduction
 - (MATH-lighteval: $\approx 24.39\%$; Easy2Hard-Bench: $\approx 21.4\%$)

10 Overall Model Comparison

To summarize the results across all approaches, we present a side-by-side comparison of the best representative model from each category on each dataset. For the baseline, we include the Description Length model, which uses the same input feature as our main models. For TF-IDF and linguistic feature models, we pick the best-performing variant found in previous sections. Transformer models are included only for individual datasets, since we did not train them on the combined dataset.

Table 17: Best model per category on MATH-lighteval

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Description Length	0.059	0.313	0.061	0.313
TF-IDF + Ridge (a)	0.627	0.158	0.328	0.212
LingFeature + Grad. Boosting	0.547	0.174	0.297	0.217
BERT-base (1st attempt)	0.670	0.148	0.404	0.199

Table 18: Best model per category on Easy2Hard-Bench

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Description Length	0.001	0.244	-0.005	0.246
TF-IDF + Ridge (a)	0.815	0.105	0.609	0.153
LingFeature + Grad. Boosting	0.606	0.153	0.457	0.180
BERT-base (1st attempt)	0.877	0.086	0.738	0.125

Table 19: Best model per category on Combined dataset

Model	Train		Test	
	R^2	RMSE	R^2	RMSE
Description Length	0.011	0.262	0.018	0.263
TF-IDF + Ridge (c)	0.592	0.168	0.228	0.233
LingFeature + Grad. Boosting	0.446	0.196	0.183	0.240

11 Conclusion

In this project, we studied the problem of predicting the difficulty of mathematical problems from their textual descriptions using a range of machine learning approaches. We evaluated simple baselines, classical text-based models, feature-engineered methods, and transformer-based models across two publicly available datasets with different difficulty annotation schemes. By normalizing difficulty labels to a unified continuous scale, we were able to compare models consistently and also explore training on a combined dataset.

Our experiments show that even relatively simple text representations provide strong performance. TF-IDF combined with Ridge regression consistently outperformed baseline models on both datasets, achieving substantial error reductions while remaining computationally efficient and robust. Retaining mathematical notation in the input proved beneficial, indicating that symbols, expressions,

and structured mathematical language carry important information for difficulty estimation.

Linguistic feature-based models using tree-based regressors offered interpretability and competitive performance, but they generally lagged behind TF-IDF approaches. However, feature importance analysis gave us some insights into which textual characteristics correlate with problem difficulty the most, such as variable counts and readability metrics.

Transformer-based models achieved the best overall performance, particularly on the Easy2Hard-Bench dataset, demonstrating their ability to capture deeper semantic. However, their gains over TF-IDF models were moderate and came at significantly higher computational cost. Moreover, fine-tuning required careful regularization to mitigate overfitting, and domain-specific pretraining (MathBERT) did not consistently outperform a general-purpose model (BERT-base), likely due to dataset size and domain mismatch.

Training on the combined dataset led to a noticeable drop in performance for all non-transformer models, suggesting that differences in difficulty definitions, distributions, and problem styles introduce additional variance that is not fully captured by simple normalization.

We could observe that each model performed significantly better on the Easy2Hard-Bench despite its smaller size. This was probably caused by already “normalized” and continuous difficulty, which was more suitable for the regression task. On the other hand, MATH-lighteval had more diverse problems together with discrete difficulty levels. This naturally caused higher prediction errors. For future work, it would be interesting to try the MATH-lighteval dataset with classification models instead of regression ones.

Overall, while the error reductions of the models were not remarkable, we still consider this project to be successful. The models are able to capture non-trivial signals related to problem difficulty despite the abstract and highly structured nature of mathematical reasoning.

References

- [1] Shivam B. and contributors. *textstat: Readability and Text Statistics for Python*. Version 0.7.12. URL: <https://github.com/textstat/textstat>.
- [2] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *CoRR* abs/1810.04805 (2018). arXiv: 1810.04805. URL: <http://arxiv.org/abs/1810.04805>.
- [3] Mucong Ding et al. “Easy2Hard-Bench: Standardized Difficulty Labels for Profiling LLM Performance and Generalization”. In: *arXiv preprint arXiv:2409.18433* (2024).
- [4] Charles R. Harris et al. *NumPy*. Version 1.26.4. URL: <https://numpy.org>.
- [5] Dan Hendrycks et al. “Measuring Mathematical Problem Solving With the MATH Dataset”. In: *arXiv preprint arXiv:2103.03874* (2021).

- [6] Adam Paszke, Sam Gross, Francisco Massa, et al. *PyTorch*. Version 2.9.1. URL: <https://github.com/pytorch/pytorch>.
- [7] Fabian Pedregosa et al. *scikit-learn: Machine Learning in Python*. Version 1.7.2. URL: <https://scikit-learn.org>.
- [8] tbs17. *tbs17/MathBERT*. URL: <https://huggingface.co/tbs17/MathBERT>.
- [9] Thomas Wolf et al. *Transformers: State-of-the-Art Natural Language Processing*. Version 4.57.3. URL: <https://github.com/huggingface/transformers>.